

Complete MDP Formulation (Revised): LowLevelAUV2DEnvV2 with RL Decision Layer and PID Control Layer

Abstract

This document gives a full, code-aligned MDP formulation for the environment defined by `core/environment/low_level_env_2d_v2.py`. The previous interpretation is corrected: in this setting, the learned policy is the *decision layer* (6 discrete actions), while the true low-level control is handled by `LowLevelAUVEnv` + `IntegratedController` (dual-loop PID + thrust allocation) + full AUV dynamics. We provide exact state/action semantics, transition decomposition, reward/termination equations, and PPO objective.

1 Scope and Corrected Layer Roles

1.1 Primary code path

This formulation is based on the following exact path:

- Wrapper env: `core/environment/low_level_env_2d_v2.py`
- Inner env: `core/environment/low_level_env.py`
- Controller: `core/control/dual_loop_pid.py` (`IntegratedController`)
- Dynamics: `core/dynamics/auv_dynamics.py` (called by inner env)

1.2 Correct layer interpretation

For `LowLevelAUV2DEnvV2`:

- **Learned layer (policy layer):** RL chooses one of 6 semantic actions.
- **Control layer (true low-level):** `IntegratedController.compute_thrust` converts action + state into 8-thruster command via velocity loop + attitude dual-loop PID + thrust allocation.
- **Plant layer:** full AUV dynamics evolves state under thrust and disturbance.

Hence this is a *single-agent MDP with hierarchical control implementation*, not a two-policy HRL (no separate learned high-level policy in this file).

2 Environment MDP Definition

Define

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma, T_{\max}). \quad (1)$$

2.1 Action space (outer RL, 6 actions)

In `LowLevelAUV2DEnvV2`,

$$\mathcal{A} = \{0, 1, 2, 3, 4, 5\}. \quad (2)$$

Semantics:

0 : forward, 1 : backward, 2 : left, 3 : right, 4 : yaw_left, 5 : yaw_right.

This action is mapped to inner action set (13-action vocabulary in `LowLevelAUVEnv`):

$$\phi(a) = \begin{cases} 1, & a = 0 \\ 2, & a = 1 \\ 3, & a = 2 \\ 4, & a = 3 \\ 7, & a = 4 \\ 8, & a = 5 \end{cases} \quad (3)$$

(i.e., surge/sway/yaw subset only).

2.2 State and observation

Underlying physical state comes from inner env dynamics:

$$s_t = (\eta_t, \nu_t, \eta_t^{\text{tar}}, t), \quad (4)$$

where

$$\eta_t = [x, y, z, \phi, \theta, \psi], \quad \nu_t = [u, v, w, p, q, r],$$

and target state η_t^{tar} is sampled in 2D plane (fixed $z = \text{fixed_z}$, zero target orientation in this wrapper).

Wrapper enforces 2D constraints after each inner step:

$$z = \text{fixed_z}, \phi = 0, \theta = 0, w = 0, p = 0, q = 0.$$

Policy observation is 8D (explicitly in `_get_observation`):

$$o_t = [e_x^b/5, e_y^b/5, e_\psi/\pi, \alpha_{tar}/\pi, u/2, v/2, r/2, d/5], \quad (5)$$

where:

$$\mathbf{e}^w = [x_{tar} - x, y_{tar} - y]^\top, \quad d = \|\mathbf{e}^w\|, \quad (6)$$

$$\alpha_{tar} = \text{atan2}(e_y^w, e_x^w), \quad (7)$$

$$e_\psi = \text{wrap}(\alpha_{tar} - \psi), \quad (8)$$

$$\begin{bmatrix} e_x^b \\ e_y^b \end{bmatrix} = \begin{bmatrix} \cos \psi & \sin \psi \\ -\sin \psi & \cos \psi \end{bmatrix} \begin{bmatrix} e_x^w \\ e_y^w \end{bmatrix}. \quad (9)$$

3 Transition Kernel Decomposition

Transition is composed of four deterministic modules + stochastic reset/external disturbance terms.

Given wrapper action a_t :

1. **Action remap:** $a'_t = \phi(a_t)$.
2. **Inner control synthesis** (in `LowLevelAUVEnv.step`):

$$\mathbf{T}_t = \Pi_{ctrl}(a'_t, \eta_t, \nu_t, \eta_t^{tar}), \quad (10)$$

where $\Pi_{ctrl} = \text{IntegratedController.compute_thrust}$ outputs 8-thruster command.

3. **Dynamics propagation:**

$$(\eta_{t+1}, \nu_{t+1}) = f_{dyn}(\eta_t, \nu_t, \mathbf{T}_t, \mathbf{f}_t^{ext}). \quad (11)$$

4. **2D projection:** enforce fixed-z and zero roll/pitch and associated rates.

Hence kernel can be written as

$$P(s_{t+1}|s_t, a_t) = \int \delta(s_{t+1} - \mathcal{F}(s_t, a_t, \xi_t)) p(\xi_t) d\xi_t, \quad (12)$$

with ξ_t covering disturbances / randomization terms.

4 Low-level controller equations actually used

4.1 IntegratedController translational loop

For translation actions under `translation_mode=distance`, action command is small displacement $\Delta \mathbf{p}_{cmd}$, converted to desired body velocity:

$$\mathbf{v}_{des} = \frac{\Delta \mathbf{p}_{cmd}}{T_s} \cdot s_{slow}, \quad (13)$$

where $s_{slow} \in [s_{min}, 1]$ near target. Then force command:

$$\mathbf{f}_{cmd} = K_p^{vel}(\mathbf{v}_{des} - \mathbf{v}_{body}) - K_d^{vel} \mathbf{v}_{body}, \quad (14)$$

with axis-wise clipping.

4.2 Attitude dual-loop PID

Attitude target $[\psi^*, \theta^*, \phi^*]$ is generated from action and heading mode. Dual-loop equations (from `DualLoopPIDController.compute`):

$$\boldsymbol{\omega}_{ref} = K_p^{out} \mathbf{e}_{ang} + K_i^{out} \int \mathbf{e}_{ang} dt, \quad (15)$$

$$\boldsymbol{\tau}_{PID} = K_p^{in} \mathbf{e}_{\omega} + K_d^{in} \dot{\mathbf{e}}_{\omega}, \quad (16)$$

$$\boldsymbol{\tau}_{FF} = J \dot{\boldsymbol{\omega}}_{ref} + B \boldsymbol{\omega} + D |\boldsymbol{\omega}| \odot \boldsymbol{\omega}, \quad (17)$$

$$\boldsymbol{\tau}_{cmd} = \boldsymbol{\tau}_{PID} + \boldsymbol{\tau}_{FF} - \boldsymbol{\tau}_{ESO}. \quad (18)$$

Then thrust allocation maps force+torque to 8-thruster command $\mathbf{T}_t$.

5 Reward model in `low_level_env_2d_v2.py`

Wrapper reward (not the inner reward) is:

$$r_t = r_{pos} + r_{imp} + r_{head} + r_{dir} + r_{prox} + r_{goal} + r_{time} + r_{spin}. \quad (19)$$

Components:

$$r_{pos} = -0.3 d_t, \quad (20)$$

$$r_{imp} = 10(d_{t-1} - d_t), \quad (21)$$

$$r_{head} = 0.3 \left(1 - \frac{|e_\psi|}{\pi} \right), \quad (22)$$

$$r_{dir} = \begin{cases} 0.5 u_t, & |e_\psi| < 0.5, \ u_t > 0, \ d_t > 0.1, \\ 0, & \text{otherwise,} \end{cases} \quad (23)$$

$$r_{prox} = 3(0.3 - d_t) \mathbb{I}[d_t < 0.3], \quad (24)$$

$$r_{goal} = 100 \mathbb{I}[d_t < 0.2], \quad (25)$$

$$r_{time} = -0.02, \quad (26)$$

$$r_{spin} = -0.1|r_t| \mathbb{I}[|r_t| > 0.5 \ \wedge \ |u_t| < 0.1]. \quad (27)$$

6 Termination in `low_level_env_2d_v2.py`

$$\text{terminated} = \mathbb{I}[d_t < 0.2] \vee \mathbb{I}[|x_t| > 20 \vee |y_t| > 20], \quad (28)$$

$$\text{truncated} = \mathbb{I}[\text{inner.current_step} \geq 90/\Delta t]. \quad (29)$$

7 Objective and PPO optimization

Policy $\pi_\theta(a|o)$ is trained with PPO (see `training/train_low_level_2d_v2.py`).

Return and value:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}, \quad V^\pi(s) = \mathbb{E}[G_t | s_t = s]. \quad (30)$$

GAE:

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t), \quad \hat{A}_t = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}. \quad (31)$$

PPO clipped objective:

$$\mathcal{L}_{clip} = \mathbb{E} \left[\min \left(\rho_t \hat{A}_t, \text{clip}(\rho_t, 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right], \quad (32)$$

$$\rho_t = \frac{\pi_\theta(a_t|o_t)}{\pi_{\theta_{old}}(a_t|o_t)}. \quad (33)$$

Overall minimized loss:

$$\mathcal{J} = -\mathcal{L}_{clip} + c_v \mathbb{E}(V_\phi - \hat{R})^2 - c_e \mathbb{E} \mathcal{H}(\pi_\theta). \quad (34)$$

8 What is “high-level” and “low-level” here (final clarification)

- **In this file pair (V2 wrapper + inner env):**
 - “high-level” = RL discrete decision over 6 actions.
 - “low-level” = controller+dynamics execution stack (IntegratedController + PID + allocator + AUV dynamics).
- **Not present here:** a second learned policy layer doing temporal abstraction (macro-action planner).

9 Code-to-equation mapping table

Equation block	Source implementation
Action map $\phi(a)$	<code>LowLevelAUV2DEnvV2.action_map</code>
8D observation formula	<code>LowLevelAUV2DEnvV2._get_observation</code>
Wrapper reward terms	<code>LowLevelAUV2DEnvV2._compute_reward</code>
Wrapper termination	<code>LowLevelAUV2DEnvV2._check_termination</code>
RL action to thrust pipeline	<code>LowLevelAUVEnv.step</code>
Velocity/goal slowdown loop	<code>IntegratedController.compute_thrust</code>
Attitude dual-loop PID	<code>DualLoopPIDController.compute</code>
Plant transition	<code>AUVDynamics.step</code> called inside <code>LowLevelAUVEnv.step</code>
PPO objective	<code>training/train_low_level_2d_v2.py</code>

Compile

From docs/:

```
pdflatex hrl_mdp_pid_rl_complete.tex
```